what do i need to use mexcuda in matlab bc currently it's giving me the error:

> In mexcuda (line 113) Trying MEX options 'E:\MATLAB\toolbox\parallel\gpu\extern\src\mex\win64\nvcc_msvcpp2019.xml'...FAILED Trying MEX options 'E:\MATLAB\toolbox\parallel\gpu\extern\src\mex\win64\nvcc_msvcpp2017.xml'...FAILED Trying MEX options 'E:\MATLAB\toolbox\parallel\gpu\extern\src\mex\win64\nvcc_msvcpp2022.xml'...FAILED Warning: No supported host compiler found, or other problem with the environment. Continuing with selected compiler to provide detailed diagnosis. > In mexcuda (line 154) mex -largeArrayDims -f E:\MATLAB\toolbox\parallel\gpu\extern\src\mex\win64\nvcc_msvcpp2019.xml NVCC_FLAGS="" -v Verbose mode is on. Error using mex Not enough input arguments. Error in mexcuda (line 185) [varargout{1:nargout}] = mex(mexArguments{:});

To use `mexcuda` in MATLAB on Windows, you need **three things properly installed and configured**:

✅ 1. A Supported Microsoft C++ Compiler

Your error:

> *No supported host compiler found*

means **MATLAB cannot find a supported Visual Studio C++ compiler**.

You need:

- **Microsoft Visual Studio** (NOT just VS Code)
- With **Desktop development with C++** workload installed

Install one of these supported versions (depending on your MATLAB release):

- Microsoft Visual Studio 2019
- Microsoft Visual Studio 2022

⚠️ Make sure during installation you select:

- ✔️ MSVC v142 or v143 toolset
- ✔️ Windows 10/11 SDK

✅ 2. CUDA Toolkit (Matching Your MATLAB Version)

You also need the **NVIDIA CUDA Toolkit** version supported by your MATLAB release.

Install from:

- NVIDIA CUDA Toolkit

⚠️ Important:
Each MATLAB version only supports specific CUDA versions.

In MATLAB, check compatibility:

```
MATLAB
```

Or check officially:

- https://www.mathworks.com/support/requirements/gpu.html ↗

If CUDA is installed but not supported, `mexcuda` fails.

✅ 3. Configure the Compiler in MATLAB

After installing Visual Studio, run:

```
MATLAB
```

Then:

MATLAB

You should see something like:

Code

If it still fails, restart MATLAB after installing Visual Studio.

🔎 Why You're Getting "Not enough input arguments"

This happens because:

- The compiler detection failed
- `mexcuda` tried to fall back to `mex`
- The environment is incomplete

The **real problem is the missing host compiler**, not your CUDA code.

✅ Quick Checklist

Requirement	Installed?
Visual Studio (full IDE)	☐
C++ workload selected	☐
CUDA Toolkit (supported version)	☐
`mex -setup C++` works	☐
Restarted MATLAB after install	☐

🛠️ If You Want Me to Be Precise

Tell me:

1. Your MATLAB version (`ver`)
2. Output of:

```
MATLAB
```

3. Output of:

```
MATLAB
```

Then I can tell you **exactly** what version of CUDA and Visual Studio you need.

if matlab is currently running a function can i open a separate instance and check for gpudevice or no

Yes — **you can open a separate instance of** MATLAB **while another one is running** and check `gpuDevice` .

They run as completely separate processes, so:

- ✅ You can open a second MATLAB window
- ✅ You can run `gpuDevice` there
- ✅ It will not stop the first one

⚠️ Important GPU Caveat

If the first MATLAB session is **actively using the GPU**, the second instance:

- Will still detect the GPU
- May show reduced available memory

- Might not be able to allocate large GPU arrays
- Could throw "out of memory" if the first session is using most VRAM

But simply running:

MATLAB

is completely safe.

🔍 What You'll See

In the second instance:

MATLAB

Will show:

- GPU name
- Compute capability
- Total memory
- Available memory
- CUDA driver/runtime versions

That's enough to verify your CUDA/MEX setup without interrupting your running job.

💡 Alternative (If You Don't Want Two Instances)

If you're just checking compatibility, you can also run:

MATLAB

That won't allocate memory.

If you'd like, tell me what your long-running function is doing (training? simulation? custom CUDA kernel?) and I can tell you whether opening another instance might impact performance.

when i run mexcuda -setup I get 'MinGW64 Compiler (C)', and when i run mex -setup C++ i get 'MinGW64 Compiler (C++)' and not visual studio but this is probably something i can just configure right

also full disclosure there's two different parts of my code - one part uses a cpp kernel and the other uses a cuda kernel

Good — this explains your error exactly.

Short answer: **MinGW will not work with `mexcuda` on Windows.**
You must use **Microsoft Visual Studio (MSVC)** as the host compiler.

🚨 Why MinGW Is The Problem

When you run:

Code

```
```

and it selects:

```
MinGW64 Compiler (C++)
```

that works for **regular C++ MEX files**, but `mexcuda` **requires MSVC on Windows**.

`mexcuda` compiles CUDA like this:

Code

MinGW is not supported as the CUDA host compiler on Windows.

So yes — this is configurable — but only if **Visual Studio with C++ tools is installed**.

✅ What You Need To Do

Install:

👉 Microsoft Visual Studio 2022

(or 2019 if your MATLAB version prefers it)

During installation, select:

- ✔ Desktop development with C++
- ✔ MSVC v143 toolset (or v142 for 2019)
- ✔ Windows 10/11 SDK

✅ After Installing Visual Studio

Restart MATLAB completely.

Then run:

MATLAB

You should now see something like:

Code

Then:

MATLAB

It should now pick something like:

Code

🔬 About Your Two Kernels

You said:

> one part uses a cpp kernel
> the other uses a cuda kernel

That's totally fine.

Once MSVC is installed:

- `mex` → will use MSVC for your C++ kernel
- `mexcuda` → will use MSVC + CUDA for your CUDA kernel

Everything will be consistent.

❗ Why MATLAB Picked MinGW

MATLAB automatically picks:

1. MSVC (if found)
2. Otherwise MinGW

Since it didn't detect Visual Studio, it fell back to MinGW.

🎯 Quick Sanity Check After Install

Run:

MATLAB

You should see **Visual Studio listed as installed**.

⚠️ One More Important Question

What MATLAB version are you using?

Because:

- Each MATLAB release supports specific versions of
  - NVIDIA CUDA Toolkit
  - Visual Studio

Tell me your MATLAB version and I'll tell you exactly which CUDA + MSVC combo is guaranteed to work.

im using matlab 2023, and i did download visual studios build tools 2022 but it's not recognizing it

Perfect — MATLAB 2023 is totally compatible with **VS 2022**, so this is fixable.

When MATLAB doesn't recognize **Visual Studio Build Tools 2022**, it's almost always one of these three issues:

✅ 1️⃣ You Installed Build Tools, But Not the Right Workload

You must have installed:

Desktop development with C++

Inside that workload, make sure these are checked:

- ✔ MSVC v143 toolset
- ✔ Windows 10 SDK (or Windows 11 SDK)

If you only installed “Build Tools” without the C++ workload, MATLAB won’t detect it.

You can modify the install by reopening:

Microsoft Visual Studio 2022 → Modify → Add “Desktop development with C++”

✅ 2️⃣ You Installed It While MATLAB Was Open

MATLAB only scans for compilers at startup.

After installing Build Tools:

Completely close MATLAB
Then reopen it and run:

MATLAB

✅ 3️⃣ MATLAB 2023 vs CUDA Compatibility

Now important question:

Which exact release?

- 2023a
- 2023b

Because MATLAB 2023 supports specific CUDA versions from the
NVIDIA CUDA Toolkit

If CUDA is installed but unsupported, `mexcuda -setup` can fail silently and fall back.

🔎 Let's Diagnose Properly

In MATLAB, run:

MATLAB

If Visual Studio is installed correctly, you should see something like:

Code

If you **don't**, then MATLAB literally cannot see the MSVC toolchain.

🚨 Common Hidden Problem

Sometimes users install:

"Build Tools for Visual Studio 2022"

but **don't install the full MSVC compiler component**.

To verify installation manually:

Check this folder exists:

Code

If that folder is missing → the compiler was not installed.

💡 If It Still Doesn't Show

Try forcing detection:

MATLAB

That prints verbose detection info.

🎯 Quick Direct Question

When you installed Build Tools, did you explicitly select:

- ✔ Desktop development with C++
- ✔ MSVC v143

Or did you just click install with defaults?

Tell me:

- 2023a or 2023b
- Whether that MSVC folder exists
- Output of `mex.getCompilerConfigurations('C++','Installed')`

We'll fix this in one pass.

does it have something to with the fact that it said my license key has expired

Short answer: **No — an expired license does NOT cause the compiler detection problem.**

The fact that:

Code

is selecting **MinGW64** means MATLAB is running and functioning normally. If your license were truly expired, MATLAB would:

- Refuse to start, **or**

- Immediately block most functionality

It would not silently hide Visual Studio.

🔎 What the License *Could* Affect

There is only one license-related edge case:

If your **Parallel Computing Toolbox** license is expired, then:

- `mex` (C++) → still works
- `mexcuda` → may error
- `gpuDevice` → may error

Because `mexcuda` belongs to the Parallel Computing Toolbox.

You can check with:

MATLAB

Look for:

Code

If it's missing or says expired, that's relevant.

🚨 But This Is NOT a License Symptom

Your specific error:

> No supported host compiler found

is purely a **compiler detection issue**.

That means:

- Visual Studio C++ workload not installed correctly
- MSVC toolset missing
- MATLAB not restarted after install
- Or MATLAB cannot see the VS installation path

🎯 Let's Separate the Two Issues

Please run these three commands and paste output:

MATLAB

MATLAB

MATLAB

That will immediately tell us:

- Is licensing okay?
- Is Visual Studio detected?
- Is CUDA working?

If I had to bet based on everything you've described:

👉 It's 95% a missing **MSVC v143 toolset** inside Visual Studio Build Tools.

License expiration almost never causes this exact compiler fallback behavior.

We'll get this working.

no the visual studio license has expired

Ah — that's different.

If your **Visual Studio license expired**, here's the key point:

✅ It will NOT prevent MSVC from working for MATLAB

The C++ compiler used by:

- `mex`
- `mexcuda`
- MATLAB

comes from the **MSVC Build Tools**, not from the Visual Studio IDE license status.

Even if:

- The Visual Studio IDE says "license expired"
- You can't use the GUI

👉 The **C++ toolchain still works** as long as it's installed.

🎯 Important Distinction

There are three separate things:

1. Visual Studio IDE (the editor) license
2. Visual Studio Build Tools (compiler toolchain)
3. MSVC v143 components

MATLAB only cares about **#2 and #3**.

The IDE license being expired does NOT block:

```
cl.exe
```

‹/› **Code**

```
MSVC toolchain
```

🔎 What's Actually Happening

If MATLAB still shows:

Code

then one of these is true:

- MSVC v143 toolset was never installed
- The installation is incomplete
- The environment variables are broken
- MATLAB hasn't been restarted
- The compiler folder isn't where MATLAB expects it

Not a license issue.

✅ Let's Verify the Compiler Exists

Check this folder on your machine:

Code

Do you see a version-number folder inside (like 14.3x.xxxxx)?

If that folder exists → the compiler is installed
If it does NOT → the C++ workload wasn't installed

💡 Also Important

Did you install:

- Visual Studio Build Tools 2022

  or

- Full Microsoft Visual Studio 2022

Either is fine — but the **C++ workload must be selected**.

🚀 Quick Test Outside MATLAB

Open **Command Prompt** and type:

Code

If you see:

Code

then MSVC works.

If you get:

Code

then the toolchain isn't properly installed.

🧠 Bottom Line

An expired Visual Studio IDE license:

- ❌ Does NOT break mex
- ❌ Does NOT break mexcuda
- ❌ Does NOT stop MSVC
- ❌ Is NOT your issue

This is still a toolchain detection problem.

If you want, tell me:

- Does `cl` work in Command Prompt?
- Does that MSVC folder exist?
- Are you on 2023a or 2023b?

We'll narrow this down fast.

C:\Program Files\Microsoft Visual Studio\2022\BuildTools\VC\Tools\MSVC\

where do i run this line of code

That line is **not code** — it's a **folder path** on your computer.

You don't run it. You check whether it exists.

✅ How to Check It

Method 1 (Easiest)

1. Open **File Explorer**
2. Copy and paste this into the address bar:

Code

```
C:\Program Files\Microsoft Visual Studio\2022\BuildTools\VC\Tools\MSVC\
```

3. Press Enter

If everything is installed correctly, you should see a folder with a version number like:

Code

If the folder:

- ✅ Exists → MSVC is installed
- ❌ Says "Location not found" → the C++ toolchain was never installed

---

✅ Alternative: Check via Command Prompt

1. Press **Win + R**
2. Type `cmd`
3. Run:

Code

If it lists version folders → good
If it says "File Not Found" → toolchain missing

🎯 What This Tells Us

If that folder doesn't exist, you need to:

Open the Visual Studio Installer → Modify →
Select:

- ✔️ Desktop development with C++
- ✔️ MSVC v143
- ✔️ Windows SDK

This has nothing to do with your license being expired.

Once you check that folder, tell me what you see — that immediately determines the next step.

i dont see build tools in there but im pretty sure it's been installed? im not entirely sure

Good — this actually narrows it down a lot.

If you **don't see the** `BuildTools` **folder**, then one of these is true:

1. You installed the full IDE instead of Build Tools
2. You installed Build Tools to a different path
3. The C++ workload was never installed

Let's check properly.

✅ Step 1 — Check What's Actually Installed

Open:

Visual Studio Installer
(Press Start → type *Visual Studio Installer*)

You should see something like:

- Microsoft Visual Studio 2022
  or
- Visual Studio Build Tools 2022

Do you see one of those listed?

✅ Step 2 — If You See "Visual Studio 2022"

Then your compiler path is probably:

```
id="gklm8n"
```

(or `Professional` instead of `Community`)

👉 Check that path instead.

✅ Step 3 — If You See "Build Tools 2022"

Then the path should be:

```
id="nx7m3l"
```

If that folder doesn't exist, the C++ workload was not installed.

🚨 The Most Common Mistake

People install Visual Studio but **don't select**:

✔ Desktop development with C++

Without that, there is **no compiler**, even though VS is installed.

🎯 The Real Test (Most Important)

Open Command Prompt and run:

```
id="r2v6zp"
```

If you get a path like:

```
id="z1w5qr"
```

→ The compiler exists.

If you get:

```
id="e7y3lm"
INFO: Could not find files
```

→ The compiler is not installed.

💡 Don't Worry

This is extremely common.
95% chance you just need to:

Visual Studio Installer → Modify →
Add:

- Desktop development with C++
- MSVC v143
- Windows SDK

Then restart MATLAB and run:

```
MATLAB
mex -setup C++
```

Tell me what you see in the Visual Studio Installer window — that's the key piece now.

step 2 checks out for me, it is in the professional -> vc -> tools folder

Perfect — that explains a lot.

Since you **found the MSVC toolchain** under:

```
C:\Program Files\Microsoft Visual Studio\2022\Professional\VC\Tools\MSVC\
```

...that means the compiler **is installed**, but MATLAB isn't detecting it because it's looking in the **BuildTools path by default**.

✅ How to Fix MATLAB Detection

You have two options:

Option 1 — Use `mex -setup` with Manual Path

You can manually tell MATLAB where MSVC is. For example:

1. Open MATLAB.
2. Run:

```
mex -setup -v C++
```

This will show **all detected compilers**. If it doesn't automatically pick `Professional`, you can **create a custom XML MEX config** pointing to:

```
C:\Program Files\Microsoft Visual Studio\2022\Professional\VC\Tools\MSVC\<version
```

MATLAB 2023a/b usually auto-detects MSVC in `Professional` / `Community` installations, but if it doesn't, this is the fallback.

Option 2 — Add VS to the Environment (Easiest)

1. Open **x64 Native Tools Command Prompt for VS 2022**
   (Start → Visual Studio 2022 → Developer Command Prompt → x64 Native Tools)
2. Type `matlab` to launch MATLAB from this prompt.

Because `cl.exe` and all paths are now in the environment, `mex -setup` will **definitely detect Visual Studio**.

✅ After Detection

- Run:

```MATLAB
mex -setup C++
mexcuda -setup
```

You should now see something like:

```Code
Microsoft Visual C++ 2022
nvcc_msvcpp2022.xml
```

- `mex` → for C++ MEX
- `mexcuda` → for CUDA kernels

💡 Note About Your Two Kernels

Now that MATLAB sees MSVC:

- Your **C++ kernel** will compile with `mex`
- Your **CUDA kernel** will compile with `mexcuda`
- The expired **Visual Studio IDE license** does **not matter** at all — the compiler works fine.

If you want, I can write **exact step-by-step commands to get `mexcuda` to work immediately with your VS Professional installation**, so you can just copy/paste in MATLAB.

Do you want me to do that?

after running mex -setup -v C++

i got

Looking for compiler 'Microsoft Visual C++ 2022' ...
... Looking for environment variable 'ProgramFiles(x86)' ...Yes ('C:\Program Files (x86)').
... Looking for file 'C:\Program Files (x86)\Microsoft Visual Studio\Installer\vswhere.exe' ...Yes.
... Executing command '"C:\Program Files (x86)\Microsoft Visual Studio\Installer\vswhere.exe" -version "[17.0,18.0)" -products Microsoft.VisualStudio.Product.Enterprise -property installationPath -format value' ...No.
... Looking for environment variable 'ProgramFiles(x86)' ...Yes ('C:\Program Files (x86)').
... Looking for file 'C:\Program Files (x86)\Microsoft Visual Studio\Installer\vswhere.exe' ...Yes.
... Executing command '"C:\Program Files (x86)\Microsoft Visual Studio\Installer\vswhere.exe" -version "[17.0,18.0)" -products Microsoft.VisualStudio.Product.Professional -property installationPath -format value' ...No.
... Looking for environment variable 'ProgramFiles(x86)' ...Yes ('C:\Program Files (x86)').
... Looking for file 'C:\Program Files (x86)\Microsoft Visual Studio\Installer\vswhere.exe' ...Yes.
... Executing command '"C:\Program Files (x86)\Microsoft Visual Studio\Installer\vswhere.exe" -version "[17.0,18.0)" -products Microsoft.VisualStudio.Product.Community -property installationPath -format value' ...No.

```
Did not find installed compiler 'Microsoft Visual C++
2022'.
... Looking for compiler 'MinGW64 Compiler (C++)' ...
... Looking for environment variable 'MW_MINGW64_LOC'
...Yes
('C:\ProgramData\MATLAB\SupportPackages\R2023a\3P.in
strset\mingw_w64.instrset').
... Looking for file
'C:\ProgramData\MATLAB\SupportPackages\R2023a\3P.ins
trset\mingw_w64.instrset\bin\g++.exe' ...Yes.
... Looking for folder
'C:\ProgramData\MATLAB\SupportPackages\R2023a\3P.ins
trset\mingw_w64.instrset' ...Yes.
... Looking for environment variable 'MW_MINGW64_LOC'
...Yes
('C:\ProgramData\MATLAB\SupportPackages\R2023a\3P.in
strset\mingw_w64.instrset').
... Executing command
'C:\ProgramData\MATLAB\SupportPackages\R2023a\3P.ins
trset\mingw_w64.instrset\bin\g++ -dumpmachine' ...Yes
('x86_64-w64-mingw32').
... Looking for environment variable 'MW_MINGW64_LOC'
...Yes
('C:\ProgramData\MATLAB\SupportPackages\R2023a\3P.in
strset\mingw_w64.instrset').
... Executing command
'C:\ProgramData\MATLAB\SupportPackages\R2023a\3P.ins
trset\mingw_w64.instrset\bin\g++ -dumpversion' ...Yes
('6.3.0').
Found installed compiler 'MinGW64 Compiler (C++)'.
```

Ah — now it's crystal clear what's happening. Thanks for pasting the verbose output.

Here's what MATLAB is telling us:

🔍 What the Output Means

1. MATLAB tried to detect **MSVC 2022** under **Enterprise, Professional, and Community editions** using `vswhere.exe`.

Code

```
... Executing command ... -products Microsoft.VisualStudio.Product.Professional .
```

- It **did not find any installation** via `vswhere`.
- MATLAB only detects **VS installations that are registered correctly with the VS installer**.
- Your Professional edition exists on disk, but apparently `vswhere` can't see it.

2. MATLAB then falls back to **MinGW**, which is why it says:

Code

```
Found installed compiler 'MinGW64 Compiler (C++)'.
```

✅ Why This Happens

This usually occurs when:

- You have **Visual Studio installed under a user-specific folder** rather than the default path.
- Or your **Visual Studio Professional is not registered in the VS Installer database**, which `vswhere.exe` relies on.
- MATLAB 2023 **cannot manually probe the disk** — it only uses `vswhere.exe` to detect MSVC.

💡 So MATLAB thinks "no VS installed" → falls back to MinGW.

🔧 How to Fix

You have **two options**:

Option 1 — Use the Developer Command Prompt

1. Open **x64 Native Tools Command Prompt for VS 2022**
   (Start → Visual Studio 2022 → Developer Command Prompt → x64 Native Tools)
2. Launch MATLAB from this prompt:

```cmd
matlab
```

3. Run:

```MATLAB
mex -setup C++
mexcuda -setup
```

Why it works:
The environment variables `PATH`, `INCLUDE`, `LIB` are set correctly, so MATLAB **can find** `cl.exe` even if `vswhere` fails.

Option 2 — Reinstall/Modify Visual Studio Professional

1. Open **Visual Studio Installer** → Modify →
2. Make sure **Desktop development with C++** is installed
3. Also, check **MSVC v143 toolset**
4. Apply changes, then restart MATLAB normally

Why it works:
This properly registers your Professional edition in the installer database, so `vswhere` can find it.

⚡ Recommended Approach

For a quick fix without reinstalling, **Option 1 is faster**:

- Just start MATLAB from the **VS Developer Command Prompt**
- Then run `mex -setup C++` and `mexcuda -setup`
- After this, both your **C++** and **CUDA kernels** will compile correctly

---

If you want, I can write the **exact commands to get `mexcuda` working right now** using your Professional installation, so you can just copy/paste in MATLAB.

Do you want me to do that?

can i even do all that when my license has expired

Yes — you **can still do all of that**, and here's why:

🔑 Key Distinction

1. **MATLAB license vs. Visual Studio license**
   - Your **MATLAB license** controls whether MATLAB itself runs. If MATLAB is running, your license is fine enough to use it.
   - Your **Visual Studio IDE license** affects the IDE (editor) GUI, but **not the MSVC compiler toolchain**.
2. **MSVC compiler works without the IDE license**
   - Even if the Visual Studio IDE says "License expired," the **C++ compiler (`cl.exe`) still works**.
   - MATLAB only needs the compiler binaries, **not the IDE GUI**.

✅ What You Can Do Right Now

- Open **x64 Native Tools Command Prompt for VS 2022**
- Start MATLAB from that prompt:

```cmd
matlab
```

- Run:

MATLAB

```
mex -setup C++
mexcuda -setup
```

This will detect your compiler and CUDA correctly.

- The expired IDE license will **not block compilation**.
- Your C++ kernels and CUDA kernels will work fine.

💡 Extra Note

The only scenario where an IDE license would matter is if you wanted to **open the Visual Studio GUI and edit projects**, but MATLAB doesn't need that.

If you want, I can give a **step-by-step copy/paste guide** to get `mexcuda` fully working with your current Professional installation, even with the IDE license expired. This will get both your C++ and CUDA kernels compiling immediately.

Do you want me to do that?

where is developer command prompt, bc i almost immediately get kicked out of visual studio professional 2022 when i open it

Ah — that explains why you're getting stuck. The **Professional IDE is expired**, so the main Visual Studio GUI won't run. But you **don't need the GUI** at all — you only need the **Developer Command Prompt**, which is separate and works even with an expired IDE license.

🔹 How to find and use the Developer Command Prompt

1. Press **Win + S** (or click Start)
2. Search for:

```
x64 Native Tools Command Prompt for VS 2022
```

- It should appear as a separate shortcut, **not the Visual Studio IDE**
- On most systems it's under:

```
Start → Visual Studio 2022 → Visual Studio Tools
```

3. Right-click → **Run as administrator** (optional but sometimes needed for compilation)
4. A black command prompt opens. This is your environment with `cl.exe` and all MSVC tools ready.

🔹 Launch MATLAB from it

Inside that prompt, type:

```
matlab
```

- MATLAB will inherit the environment variables (`PATH` , `INCLUDE` , `LIB`) from the Developer Command Prompt
- This allows MATLAB to detect MSVC **even if the IDE itself has an expired license**

🔹 Then configure MEX and MEXCUDA

In MATLAB (running from this prompt):

```
mex -setup C++
```

```
mexcuda -setup
```

You should now see something like:

Code

```
Microsoft Visual C++ 2022
nvcc_msvcpp2022.xml
```

- `mex` → for your C++ kernel
- `mexcuda` → for your CUDA kernel

Everything will work without touching the Visual Studio GUI.

If you want, I can give **exact step-by-step commands and paths** so you can copy/paste and have `mexcuda` working immediately with your current installation.

Do you want me to do that?